

Reviewer Pack — Minimal Local System Rank and Communication Complexity Growth...

Entry 9bd1532718c1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 21:32:22 UTC

Minimal Local System Rank and Communication Complexity Growth

Entry ID: 9bd1532718c1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 21:32:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every n -vertex communication graph G with bounded degree, the minimal local system rank ($\text{mls}(G)$) of its associated algebraic variety is linearly correlated with its communication complexity $c(G)$, such that $\text{mls}(G) = O(c(G))$.

Rationale (proposer's reasoning):

Local systems in algebraic geometry provide a rich invariant for studying geometric properties. By mapping communication graphs to algebraic varieties, we may expose hidden structural information about communication complexity. This conjecture aims to bridge the gap between algebraic geometry and computational complexity by leveraging the geometric properties of the constructed varieties.

Taxonomy category: c003b_cumulative_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e61d5db3df384bd8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of the generated random n -vertex communication graphs with bounded degree exhibit a correlation coefficient between the minimal local system rank ($\text{mls}(G)$) and communication complexity $c(G)$ greater than or equal to 0.7, calculated using Pearson's correlation on an aggregate statistic across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local system rank" AND "communication complexity" - "algebraic geometry" AND "communication graph" - "algebraic variety" AND communication graph degree bounded

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

# Helper functions for linear algebra
def matrix_multiplication(A, B):
    if len(A[0]) != len(B):
        raise ValueError("Incompatible dimensions for matrix multiplication")
    result = [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]
    return result

def gaussian_elimination(matrix):
    augmented_matrix = [row[:] + [0] for row in matrix]
    n = len(matrix)
    m = len(matrix[0])

    for i in range(n):
        # Find the pivot
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = -augmented_matrix[j][i] / augmented_matrix[i][i]
            for k in range(m):
                augmented_matrix[j][k] += factor * augmented_matrix[i][k]

    return augmented_matrix
```

```

def rank_of_matrix(matrix):
    rref = gaussian_elimination(matrix)
    rank = 0
    for row in rref:
        if any(row):
            rank += 1
    return rank

# Function to generate a random communication graph with bounded degree
def generate_communication_graph(n, max_degree):
    G = {i: [] for i in range(n)}
    edges_added = 0
    while edges_added < n * max_degree // 2:
        u, v = random.sample(range(n), 2)
        if u != v and v not in G[u]:
            G[u].append(v)
            G[v].append(u)
            edges_added += 1
    return G

# Function to compute communication complexity of a graph
def communication_complexity(G):
    n = len(G)
    total_edges = sum(len(neighbors) for neighbors in G.values()) // 2
    return total_edges * (n - 1)

# Function to map a communication graph to an algebraic variety and compute mls(G)
def minimal_local_system_rank(G):
    n = len(G)
    A = [[0] * n for _ in range(n)]

    # Create the adjacency matrix with weights
    for u, neighbors in G.items():
        for v in neighbors:
            A[u][v] += 1

    return rank_of_matrix(A)

# Function to run a single trial
def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_max = 40
    instances_tested = 0
    metric_values = []
    counterexample = ""

    for n in range(5, n_max + 1):
        G = generate_communication_graph(n, max_degree=3)
        mls_G = minimal_local_system_rank(G)
        c_G = communication_complexity(G)

        instances_tested += len(G)
        metric_values.append(mls_G / c_G)

```

```

if instances_tested < 30:
    return {
        "metric_name": "mls(G) / c(G)",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_rank = sum(metric_values) / len(metric_values)
std_dev = math.sqrt(sum((x - mean_rank) ** 2 for x in metric_values) / len(metric_values))

return {
    "metric_name": "mls(G) / c(G)",
    "metric_value": mean_rank,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": mean_rank >= 0.7,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fra
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fra
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='low_correlation' first_failing_seed={first_fai

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4a144fe.py", line 130, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4a144fe.py", line 96, in run_trial
    mls_G = minimal_local_system_rank(G)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4a144fe.py", line 83, in minimal_local
    return rank_of_matrix(A)
           ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4a144fe.py", line 48, in rank_of_matrix
    rref = gaussian_elimination(matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4a144fe.py", line 41, in gaussian_elim
    factor = -augmented_matrix[j][i] / augmented_matrix[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented the calculation of the required correlation coefficients to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for the computation of the necessary statistics.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13377
2	preregistration	ollama_remote	glm4:latest	0	0	13735
3	novelty	ollama_remote	glm4:latest	0	0	11682
4	novelty	ollama_remote	glm4:latest	0	0	21762
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14961
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21562
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18836
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20047
9	judge	ollama_remote	glm4:latest	0	0	12373

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148334 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9bd1532718c1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9bd1532718c1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9bd1532718c1.tar.gz` (if generated)